



Universidad de Concepción
Departamento de Informática y Cs. de la Computación
Ingeniería Civil Informática

Tarea 1
503302-1: TOPICOS EN MANEJO DE GRANDES
VOLUMENES DE DATOS

Gabriel Huerta Torres
Diego Oyarzo Navia

29 de Septiembre de 2025

1. Actividad 1

En esta actividad se desarrolló la infraestructura base para la detección de heavy hitters de k -mers canónicos en genomas, estableciendo un ground truth confiable que permita evaluar posteriormente el desempeño de las estructuras de sketch a implementar.

1.1. Representación y Codificación de Datos

Se implementó un sistema de codificación eficiente para bases de adn utilizando representación binaria de 2 bits por base, donde:

- A: 00 (0)
- C: 01 (1)
- G: 10 (2)
- T: 11 (3)

Esta codificación permite almacenar k -mers de hasta 32 bases en una variable `uint64_t`, optimizando significativamente el uso de memoria y las operaciones de comparación.

1.2. Procesamiento de Genomas

El procesamiento se realizó mediante las siguientes etapas:

- Se filtran las secuencias, para lo que se descartan automáticamente las bases indefinidas (N) y caracteres no válidos, manteniendo únicamente A, C, G, T.
- Se realiza una segmentación controlada y se gestiona eficientemente la memoria, para lo cual se procesa el 30 % del archivo original de un genoma, permitiendo escalabilidad en el análisis.
- Finalmente se hace una normalización, donde se ignoran las líneas de encabezado FASTA y se procesan únicamente las secuencias nucleotídicas.

1.3. Extracción de K-mers Canónicos

Para cada posición i en la secuencia, se extrae un k -mer de longitud k . El k -mer canónico se obtiene mediante el cálculo del reverso complementario, aplicando operaciones *bitwise* para obtener el complemento (XOR con `0b11`) y luego invirtiendo el orden. Después, se compara el k -mer original con su reverso complementario, seleccionando el menor en orden lexicográfico.

Este proceso garantiza una representación única por cada par k -mer / reverso-complementario, reduciendo el espacio de búsqueda y eliminando la redundancia de la doble hebra.

Para la implementación en la tarea, donde $k \in \{21, 31\}$, se procesaron **k-mer** de longitudes 21 y 31, valores comúnmente utilizados en bioinformática que proporcionan un balance entre especificidad y cobertura genómica. Finalmente cabe mencionar que se utilizó un `std::unordered_map<uint64_t, int>` para mantener conteos exactos de cada k -mer canónico en la extracción del ground truth para los HH según la definición estipulada en el enunciado de la tarea.

1.4. Análisis de Memoria

Se implementó un sistema de monitoreo que calcula la memoria utilizada por los heavy hitters, considerando las claves `uint64_t` de 8 bytes, los valores de frecuencia `int` de 4 bytes, el overhead de estructuras hash: ≈ 24 bytes por entrada, y la representación `string` para salida: k bytes por k -mer.

1.5. Resultados y Validación

En la implementación inicial, los resultados se almacenaban en formato CSV con la siguiente estructura:

```
Genoma,kmer,frecuencia,k_value,is_heavy_hitter
```

Sin embargo, en la implementación actual optimizada (hecha en la actividad 3), los k -mers y su clasificación como heavy hitters se almacenan directamente en un vector en memoria:

```
std::vector<std::pair<uint64_t, bool>> subconjunto;
```

donde cada par contiene el k -mer codificado (`uint64_t`) y un booleano que indica si es heavy hitter. Esta optimización elimina la sobrecarga de escritura/lectura de archivos CSV intermedios.

Los resultados finales de comparación entre algoritmos se almacenan en CSV con estructura:

```
Genoma,Total_kmers,Real_HH,Phi,Threshold,Algoritmo,Precision,Recall,F1_Score,TP,FP,FN,TN,Memory_MB,Mean_Abs_Error,Mean_Rel_Error
```

Para cada ejecución se registran:

- El número total de k -mers evaluados en el subconjunto.
- El número de heavy hitters reales identificados en el ground truth.
- Las métricas de performance: precisión, recall, F1-score, matriz de confusión
- El consumo de memoria de cada algoritmo sketch.
- Los errores asociados a los sketches.

El umbral $\phi = 0,0000044$ fue seleccionado tras experimentación iterativa para encontrar el balance óptimo entre detección de heavy hitters reales y control de falsos positivos.

2. Actividad 2

Se programaron tres estructuras: Count Sketch, Tower Sketch(TS CMCU) y CountMin con conservative update, donde Tower Sketch usa CountMin con CU. Cada una implementa las operaciones `insert(kmer)` y `estimar_freq(kmer)`. En la inserción se utiliza el k -mer canónico.

2.1. Count Sketch

Listing 1: Parte de la implementación Count Sketch

```
class CountSketch
{
    private:
        int d;
        int w;
        int c = 1;
        std::vector<std::vector<int64_t>>> C;
        std::vector<int> g;
        std::vector<int> init_g(int tamano);

    public:
        CountSketch(int d, int w);

        void insert(uint64_t elemento);
        double estimar_freq(uint64_t elemento);
        double get_size_MB() const;
};
// ... resto en el archivo cpp
```

2.2. CountMin CU

Listing 2: Parte de la implementación CountMin CU

```
class CountMin_CU
{
    private:
        int d;
        int w;
        std::vector<std::vector<uint64_t>>> C;

    public:
        CountMin_CU(int d, int w);

        void insertCMin(uint64_t elemento);
        double estimar_freq(uint64_t elemento) const;
        size_t get_memory_size() const;
};
// ... resto en el archivo cpp
```

2.3. TS-CMCU (Tower Sketch)

Listing 3: Parte de la implementación Tower Sketch

```
class TowerSketch
{
    private:
        int niveles;
        std::vector<CountMin_CU> Tower;
        void tower_create(int niveles);
    public:
        TowerSketch(int niveles);
        void insert(uint64_t elemento);
        double estimar_freq(uint64_t elemento);
        size_t get_memory_size() const;
};
// ... resto en el archivo cpp
```

2.4. Calibración de Sketches

Los algoritmos CountSketch (CS) y TowerSketch con CountMin y Conservative Update (TS-CMCU) fueron calibrados utilizando diferentes configuraciones de parámetros para evaluar el compromiso entre precisión y uso de memoria.

2.4.1. Parámetros de Calibración

Para ambos algoritmos se establecieron los siguientes parámetros:

- **CountSketch (CS):** Se utilizó una configuración con $d = 15$ filas y $w = 500,000$ columnas.
- **TowerSketch (TS-CMCU):** Se implementó con una arquitectura de 8 niveles, aplicando la técnica de *Conservative Update* para mejorar la precisión en la estimación de frecuencias. Cada nivel utiliza una estructura CountMin_CU con un ancho w proporcional al nivel, definido como $w = 5000 \times 2^{(n-1)}$, donde n es el número de nivel. Por ejemplo, el nivel 1 tiene $w = 5000$, el nivel 2 tiene $w = 10\,000$, y así sucesivamente hasta el nivel 8, que alcanza un ancho de $w = 640\,000$.

Cuadro 1: Configuración de parámetros para calibración de sketches

Algoritmo	Memoria (MB)	Parámetros	Observaciones
CountSketch	57.22	$d = 15, w = 500,000$	Configuración estándar
TowerSketch	67.00	8 niveles	Con Conservative Update

TowerSketch utiliza aproximadamente 17.1 % más memoria que CountSketch (9.78 MB adicionales), manteniendo ambos un uso de memoria controlado dentro del límite establecido de 1 MB para los heavy hitters detectados.

3. Actividad 3: Detección de Heavy Hitters y Validación

3.1. Metodología de Evaluación

La evaluación se realizó sobre un conjunto de 50 genomas , utilizando un umbral $\phi = 4,4 \times 10^{-6}$ para la detección de heavy hitters. Se calcularon las métricas de Precision, Recall, F1-Score, así como los errores absolutos y relativos medios de las frecuencias estimadas.

3.2. Resultados de Precision, Recall y F1-Score

Cuadro 2: Resumen de métricas de clasificación por algoritmo

Algoritmo	Precision Media	Recall Media	F1-Score Medio
CountSketch	0.2116	1.0000	0.3487
TowerSketch	0.2116	0.9979	0.3485

CountSketch mantiene un Recall perfecto (100 %), detectando todos los heavy hitters reales. TowerSketch presenta una ligera reducción en el Recall (99.79 %) debido a falsos negativos distribuidos en varios genomas.

3.3. Resultados de Errores de Frecuencia

Cuadro 3: Errores de estimación de frecuencias para heavy hitters detectados

Algoritmo	Error Absoluto Medio	Error Relativo Medio
CountSketch	18.89	3.72
TowerSketch	9.79	1.48
Mejora de TS	-48.2 %	-60.2 %

Resultado clave: TowerSketch reduce el error absoluto medio en 48.2 % y el error relativo medio en 60.2 % comparado con CountSketch, demostrando una estimación de frecuencias significativamente más precisa.

3.4. Distribución de Casos por Umbral

Cuadro 4: Distribución de genomas por umbral efectivo

Umbral	Genomas	Comportamiento	Cálculo de Errores
1	5	Todos los k-mers son HH	Calculado
2	11	Alta densidad de HH	Calculado
3	13	Selectividad moderada	Calculado
4	3	Selectividad alta	Calculado
6-7	18	Muy selectivos	Calculado

3.5. Análisis Detallado por Categoría

3.5.1. Casos donde Todos son Heavy Hitters (Umbral = 1)

Para los 5 genomas donde todos los k-mers son heavy hitters:

Cuadro 5: Métricas en casos con todos HH

Métrica	CountSketch	TowerSketch
Precision	1.0000	1.0000
Recall	1.0000	1.0000
F1-Score	1.0000	1.0000
Error Absoluto Medio	5.72	1.93
Error Relativo Medio	5.70	1.92
Mejora de TS	-66.3 %	-66.3 %

En estos casos, TowerSketch reduce los errores en aproximadamente 66 %, demostrando una ventaja clara en la estimación de frecuencias incluso cuando no hay selectividad en la detección.

3.5.2. Casos Selectivos (Umbrales 2-7)

Para los 45 genomas con detección selectiva:

Cuadro 6: Estadísticas de heavy hitters en casos selectivos

Métrica	Valor
Promedio de HH por genoma	150.4
Mínimo de HH	1
Máximo de HH	1,521
Mediana de HH	108

Cuadro 7: Errores promedio en casos selectivos

Algoritmo	Error Absoluto	Error Relativo
CountSketch	24.72	3.41
TowerSketch	12.60	1.33
Mejora de TS	-49.0 %	-61.0 %

3.6. Análisis de Falsos Negativos

TowerSketch presenta falsos negativos en varios genomas:

Cuadro 8: Distribución de falsos negativos

Métrica	Valor
Total de Falsos Negativos	Variable por genoma
Genomas con FN > 0	40 de 50 (80 %)
Rango de FN	15 - 3,548
Promedio FN por genoma afectado	815.2

Los falsos negativos sugieren que TowerSketch, al aplicar Conservative Update para reducir sobreestimaciones, puede clasificar algunos heavy hitters reales como no-HH cuando sus frecuencias estimadas quedan por debajo del umbral.

4. Conclusiones

El TowerSketch presenta menor error que el CountSketch para igual memoria.

Por otro lado, la baja precisión observada de los sketches se debe a múltiples factores, como la naturaleza desbalanceada de los datos genómicos, donde la mayoría de los k-mers son poco frecuentes y menos del 0.00003 % son verdaderamente heavy hitters, generando un desbalance extremo de clases.

Luego, existen limitaciones inherentes de CountSketch y TowerSketch, donde al ser algoritmos probabilísticos tienden a sobreestimar frecuencias. Por último, el *threshold* bajo utilizado (12 apariciones) no estaría permitiendo discriminar efectivamente entre ruido genómico y k-mers realmente frecuentes, resultando en una gran cantidad de falsos positivos.

A pesar de la baja precisión absoluta, se puede ver que TowerSketch demuestra superioridad clara sobre CountSketch, indicando significativamente mejor capacidad de discriminación en *datasets* con alta dispersión de frecuencias.